

Tugas Watermarking

By:
Fatin Sajid Darwis (18224074)

Code Using:



watermark_dct.py

Alur Sistem

Load gambar host

Gambar host adalah gambar asli yang akan diberi watermark. Jika pengguna memasukkan file gambar, sistem akan membaca gambar tersebut. Jika tidak ada gambar, sistem membuat gambar wajah sintetis.

```
def load_or_create_host(path=None, size=256):
    """Load gambar dari path atau buat sintetis."""
    if path and os.path.exists(path):
        img = cv2.imread(path)
        if img is not None:
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            img = cv2.resize(img, (size, size))
            print(f"[✓] Gambar host dimuat dari: {path}")
            return img
    print("[!] Foto tidak ditemukan – menggunakan wajah sintetis.")
    return create_sample_face(size)
```

Pada bagian ini, gambar juga diubah ukurannya menjadi **256 × 256 piksel** agar proses watermarking berjalan seragam.



Membuat watermark

Watermark yang digunakan berupa data biner sebanyak 64 bit. Pada mode binary, watermark dibuat dari nilai heksadesimal 0xDEADBEEFCAFEBAE.

```
def generate_watermark(n_bits=WATERMARK_BITS, mode="binary"):
    """
    Buat watermark:
    - 'binary' : bit {0,1} secara deterministik
    - 'random' : bit acak sepenuhnya
    """
    rng = np.random.default_rng(SEED + 1)
    if mode == "binary":
        # Pola biner deterministik (bisa diganti teks/ID)
        bits = np.array([int(b) for b in format(0xDEADBEEFCAFEBAE,
        '064b')[:n_bits]])
    else:
        bits = rng.integers(0, 2, n_bits)
    return bits.astype(np.float32)
```

Bagian ini menentukan isi watermark yang akan disisipkan ke dalam gambar.



Konversi gambar RGB ke YcrCb

Sebelum watermark disisipkan, gambar RGB diubah ke ruang warna YCrCb. Kode hanya mengambil channel Y karena channel ini menyimpan informasi terang-gelap gambar.

```
img_ycbcr = cv2.cvtColor(host_img, cv2.COLOR_RGB2YCrCb).astype(np.float32)
Y = img_ycbcr[:, :, 0]
```

Channel Y dipilih karena lebih relevan terhadap struktur visual gambar dan lebih kuat terhadap kompresi JPEG.



Transformasi DCT

Channel Y kemudian diubah dari domain piksel ke domain frekuensi menggunakan Discrete Cosine Transform atau DCT.

```
H, W = Y.shape
Y_dct = cv2.dct(Y)
```

DCT membuat gambar direpresentasikan dalam bentuk koefisien frekuensi. Pada tahap ini, watermark belum disisipkan. Sistem baru menyiapkan ruang frekuensi untuk proses embedding.



Memilih koefisien mid-frequency

Kode meratakan koefisien DCT menjadi satu dimensi, lalu memilih bagian frekuensi menengah.

```
# Pilih koefisien mid-frekuensi (hindari DC dan high-freq)
flat_dct = Y_dct.flatten()
n_total = len(flat_dct)
n_bits = len(watermark_bits)

# Koefisien indeks pertengahan untuk robustness terhadap JPEG
mid_start = n_total // 6
mid_end = n_total // 2
n_coeffs = (mid_end - mid_start) // n_bits
```

Koefisien mid-frequency dipilih karena lebih seimbang. Bagian ini tidak terlalu terlihat oleh mata, tetapi masih cukup tahan terhadap kompresi JPEG.



Membuat PN sequence

Sistem membuat Pseudo-Noise sequence atau PN sequence. Pola ini berisi nilai -1 dan +1.

```
def get_pn_sequences(n_bits, n_coeffs, seed=SEED):
    """Hasilkan n_bits PN (pseudo-noise) sequences masing-masing panjang
    n_coeffs."""
    rng = np.random.default_rng(seed)
    pn = rng.choice([-1.0, 1.0], size=(n_bits, n_coeffs))
    return pn
```

PN sequence dibuat berdasarkan SEED. Dalam kode ini, nilai seed adalah 42.

```
SEED = 42
```

Seed berfungsi sebagai kunci. Seed yang sama harus digunakan saat embedding dan extraction.



Penyisipan watermark

Watermark disisipkan ke koefisien DCT menggunakan metode additive spread spectrum. Jika bit bernilai 1, sistem menambahkan sinyal. Jika bit bernilai 0, sistem mengurangi sinyal.

```
pn_seqs = get_pn_sequences(n_bits, n_coeffs)
```

```
# Embedding
for i, bit in enumerate(watermark_bits):
    polarity = 1.0 if bit >= 0.5 else -1.0
    idx_start = mid_start + i * n_coeffs
    idx_end = idx_start + n_coeffs
    flat_dct[idx_start:idx_end] += alpha * polarity * pn_seqs[i]
```

↓

Inverse DCT

Setelah watermark masuk ke koefisien DCT, sistem mengembalikan data dari domain frekuensi ke domain piksel menggunakan inverse DCT.

```
Y_wm = cv2.idct(flat_dct.reshape(H, W))
Y_wm = np.clip(Y_wm, 0, 255)
```

Fungsi np.clip() menjaga agar nilai piksel tetap berada pada rentang valid, yaitu 0 sampai 255.

↓

Menggabungkan kembali channel gambar

Channel Y yang sudah berisi watermark dimasukkan kembali ke gambar YCrCb. Setelah itu, gambar dikonversi lagi ke RGB.

```
img_wm = img_ycbcr.copy()
img_wm[:, :, 0] = Y_wm
watermarked = cv2.cvtColor(img_wm.astype(np.uint8), cv2.COLOR_YCrCb2RGB)
return watermarked
```

Hasil dari tahap ini adalah gambar baru yang sudah mengandung watermark tersembunyi.

↓

Menyimpan gambar hasil watermark

Setelah proses embedding selesai, gambar hasil watermark disimpan dalam format JPG.

```
wm_bgr = cv2.cvtColor(wm_img, cv2.COLOR_RGB2BGR)
cv2.imwrite(out_jpg, wm_bgr, [int(cv2.IMWRITE_JPEG_QUALITY), 95])
print(f"[✓] Gambar hasil watermark disimpan : {out_jpg}")
print(f"\n--- Semua file tersimpan di: {script_dir} ---")
```

Gambar disimpan dengan kualitas JPEG sebesar 95 agar kualitas visualnya tetap baik.

↓

Ekstraksi watermark

Ekstraksi adalah proses membaca kembali watermark dari gambar. Prosesnya mirip dengan embedding. Gambar dikonversi ke YCrCb, channel Y diambil, lalu DCT diterapkan kembali. Setelah itu, sistem mengambil kembali koefisien mid-frequency yang sama dan PN sequence juga dibuat ulang dengan seed yang sama.

```
def extract_watermark(watermarked_img, n_bits=WATERMARK_BITS):
    """
    Ekstrak watermark dari gambar (mungkin sudah dikompres).
    Korelasi koefisien DCT dengan PN sequence yang sama.
```

```

"""
img_ycbcr = cv2.cvtColor(watermarked_img,
cv2.COLOR_RGB2YCrCb).astype(np.float32)
Y = img_ycbcr[:, :, 0]
H, W = Y.shape

Y_dct = cv2.dct(Y)
flat_dct = Y_dct.flatten()
n_total = len(flat_dct)

mid_start = n_total // 6
mid_end = n_total // 2
n_coeffs = (mid_end - mid_start) // n_bits

pn_seqs = get_pn_sequences(n_bits, n_coeffs)

```



Keputusan bit berdasarkan korelasi

Setiap bagian koefisien DCT dibandingkan dengan PN sequence. Sistem menghitung korelasi. Jika hasilnya positif, bit dianggap 1. Jika hasilnya negatif atau nol, bit dianggap 0.

```

extracted = np.zeros(n_bits, dtype=np.float32)
for i in range(n_bits):
    idx_start = mid_start + i * n_coeffs
    idx_end = idx_start + n_coeffs
    corr = np.dot(flat_dct[idx_start:idx_end], pn_seqs[i])
    extracted[i] = 1.0 if corr > 0 else 0.0

return extracted

```

Rumus sederhananya:

$$corr = \sum F_{received}[k] \times PN[k]$$

Keputusannya:

Jika $corr > 0$, bit = 1

Jika $corr \leq 0$, bit = 0



Kompresi JPEG untuk pengujian

Kode juga menguji apakah watermark tetap bisa dibaca setelah gambar dikompresi JPEG.

```

def jpeg_compress(img_rgb, quality):
    """Kompres gambar dengan JPEG pada quality factor tertentu, lalu decode."""
    img_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)
    encode_params = [int(cv2.IMWRITE_JPEG_QUALITY), quality]
    _, encoded = cv2.imencode('.jpg', img_bgr, encode_params)
    decoded_bgr = cv2.imdecode(encoded, cv2.IMREAD_COLOR)
    return cv2.cvtColor(decoded_bgr, cv2.COLOR_BGR2RGB)

```

Nilai kualitas JPEG yang diuji adalah:

```
QF_VALUES = [5, 10, 15, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

Semakin kecil nilai QF, semakin kuat kompresi. Semakin besar nilai QF, semakin baik kualitas gambar.



Evaluasi hasil watermark

Setelah gambar dikompresi, watermark diekstrak kembali. Hasil ekstraksi dibandingkan dengan watermark asli.

```
for qf in QF_VALUES:
    comp = jpeg_compress(wm_img, qf)
    ex = extract_watermark(comp, WATERMARK_BITS)
    b = ber(wm_bits, ex)
    n_ = nc(wm_bits, ex)
    p = psnr(wm_img, comp)
    extractable = b < 0.1
```

Kode menggunakan tiga metrik evaluasi.

BER

BER mengukur jumlah bit watermark yang salah.

```
def ber(original_bits, extracted_bits):
    """Bit Error Rate: proporsi bit yang salah."""
    errors = np.sum(original_bits != extracted_bits)
    return errors / len(original_bits)
```

NC

NC mengukur kemiripan watermark asli dengan watermark hasil ekstraksi.

```
def nc(original_bits, extracted_bits):
    """Normalized Correlation antara watermark asli dan yang diekstrak."""
    orig = 2 * original_bits - 1 # {0,1} → {-1,+1}
    extr = 2 * extracted_bits - 1
    num = np.dot(orig, extr)
    den = np.sqrt(np.dot(orig, orig) * np.dot(extr, extr) + 1e-9)
    return num / den
```

PSNR

PSNR mengukur kualitas visual gambar.

```
def psnr(img1, img2):
    """Peak Signal-to-Noise Ratio antara dua gambar."""
    mse = np.mean((img1.astype(float) - img2.astype(float)) ** 2)
    if mse < 1e-10:
        return float('inf')
    return 10 * np.log10(255 ** 2 / mse)
```

Hasil Embedding Watermark

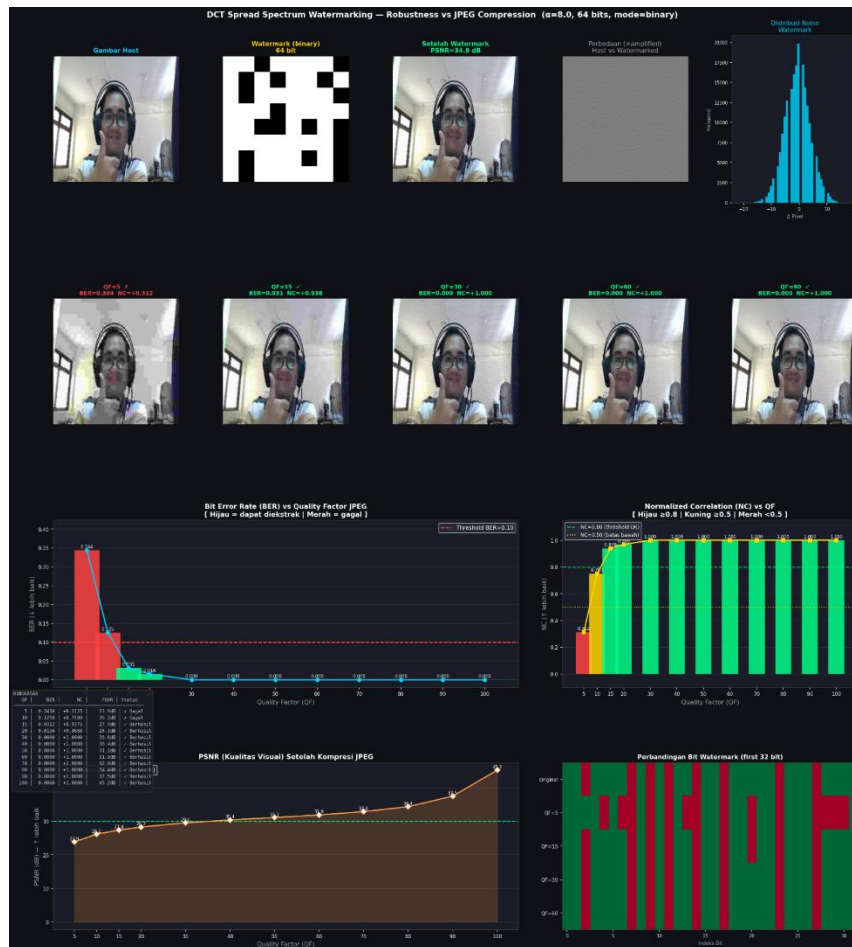
Gambar saya berhasil disisipi watermark dengan kualitas visual yang baik. PSNR antara gambar asli dan gambar watermarked adalah 34.91 dB, yang berarti perubahan piksel sangat kecil dan tidak dapat terlihat oleh mata manusia.



Berikut adalah hasil lengkap evaluasi watermark setelah kompresi JPEG dengan berbagai nilai QF:

QF	BER	NC	PSNR (dB)	Status
5	0.3438	+0.3125	23.9	Gagal
10	0.1250	+0.7500	26.2	Gagal
15	0.0312	+0.9375	27.4	Gagal
20	0.0156	+0.9688	28.3	Berhasil
30	0.0000	+1.0000	29.6	Berhasil
40	0.0000	+1.0000	30.4	Berhasil
50	0.0000	+1.0000	31.1	Berhasil
60	0.0000	+1.0000	31.9	Berhasil
70	0.0000	+1.0000	32.9	Berhasil
80	0.0000	+1.0000	34.4	Berhasil
90	0.0000	+1.0000	37.5	Berhasil
100	0.0000	+1.0000	45.2	Berhasil

Grafik Evaluasi Lengkap:



Berdasarkan hasil eksperimen, watermark TIDAK dapat diekstrak ($BER \geq 0.10$) pada nilai QF:

QF = 5 → BER = 0.3438 (34.38%-bit salah) — GAGAL

QF = 10 → BER = 0.1250 (12.50%-bit salah) — GAGAL

QF = 15 → BER = 0.0312 (3.12%-bit salah) — BATAS (NC sangat tinggi)

Mulai QF = 20, watermark berhasil diekstrak dengan BER = 0.0156 dan NC = +0.9688, yang secara praktis dianggap berhasil. Pada QF ≥ 30, seluruh 64 bit watermark terekstrak dengan sempurna (BER = 0, NC = +1.0000).

Mengapa Gagal di QF Rendah?

JPEG dengan QF rendah menerapkan kuantisasi agresif pada koefisien DCT, yaitu membagi koefisien dengan nilai matriks kuantisasi yang besar lalu membulatkan ke integer. Modifikasi kecil yang ditambahkan oleh watermark (sebesar $\alpha = 8.0$)

- Pada QF=5: kuantisasi menghapus hampir semua detail mid-frekuensi → watermark hilang
- Pada QF=10: kuantisasi masih terlalu agresif → 12.5%-bit salah
- Pada QF=15: hampir semua bit benar (3.12% error), namun masih di batas gagal
- Pada QF≥20: kuantisasi lebih lunak → modifikasi watermark bertahan

Kualitas Visual (PSNR)

PSNR gambar watermarked vs asli adalah 34.91 dB, di atas threshold 30 dB. Ini berarti perubahan visual akibat watermark tidak terlihat oleh mata manusia. Watermark bersifat invisible (tidak kasat mata).

Menarik diamati bahwa NC pada QF=15 sudah sangat tinggi (+0.9375) meskipun BER = 0.0312. Hal ini menunjukkan bahwa meskipun 2 dari 64-bit salah, secara keseluruhan watermark masih sangat berkorelasi dengan watermark asli. Dalam aplikasi praktis, error correction code (ECC) seperti Hamming Code atau Reed-Solomon dapat mengoreksi kesalahan kecil ini.